

Temperature, Worked Out by Hand

The single knob that decides whether a language model speaks with conviction or wanders — shown as a rescaling of logits before softmax, with the distribution and its entropy computed at every setting.

What this document does. It fixes one logit vector, divides it by a temperature T , and watches the softmax distribution morph from sharp to flat as T grows — computing the probabilities *and* the Shannon entropy at $T = 0.5, 1, 2, 5, 100$ by hand. It then proves the two limits ($T \rightarrow 0$ is greedy arg max, $T \rightarrow \infty$ is uniform) and explains why temperature is the cleanest diversity control we have. Numbers come from the NumPy snippet in Appendix A.

Prerequisite: the softmax PDF (#1). Temperature *is* softmax with rescaled inputs — read that first if the normaliser Z is unfamiliar.

1 The formula

Temperature inserts one division before the softmax of Module 0.3:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} \quad T > 0.$$

$T = 1$ recovers ordinary softmax. $T < 1$ *amplifies* the gaps between logits; $T > 1$ *shrinks* them. Because softmax reads only differences (shift-invariance, PDF #1), scaling every logit by $1/T$ is the same as scaling every *gap* by $1/T$ — and the gaps are precisely what decide how peaked the distribution is.

Intuition

A logit gap of Δ becomes a probability ratio of $e^{\Delta/T}$. At $T = 0.5$ a gap of 1 is felt as $e^2 = 7.4\times$; at $T = 2$ the same gap is only $e^{0.5} = 1.65\times$. Lowering T stretches molehills into mountains (the top token runs away with the mass); raising T flattens mountains into molehills (every token looks comparable). You are not changing what the model *believes* — the ranking never moves — only how strongly it *commits*.

2 Worked by hand on a five-token vector

Take the same logits as the softmax walk-through, $\mathbf{z} = (2.0, 1.0, 0.1, -1.0, 3.0)$, over the five tokens `cat`, `sat`, `on`, `the`, `mat`. For each T we divide, exponentiate, and normalise.

Hand computation at $T = 0.5$. Divide by 0.5 (i.e. *double* every logit): $\mathbf{z}/T = (4, 2, 0.2, -2, 6)$. Then

$$e^{(\cdot)} = (54.60, 7.389, 1.221, 0.135, 403.4), \quad Z = 466.8,$$

$$\mathbf{p} = (0.1170, 0.0158, 0.0026, 0.0003, 0.8643).$$

`mat` has jumped from 63% (at $T = 1$) to 86%. Halving the temperature nearly doubled the winner's lead.

The full sweep. Adding the Shannon entropy $H(\mathbf{p}) = -\sum_i p_i \log_2 p_i$ (in bits) as a single scalar summary of “how spread out”:

T	cat	sat	on	the	mat	H (bits)	$p_{\max}$
0.5	0.1170	0.0158	0.0026	0.0003	0.8643	0.6645	0.8643
1.0	0.2333	0.0858	0.0349	0.0116	0.6343	1.4542	0.6343
2.0	0.2587	0.1569	0.1001	0.0577	0.4266	2.0181	0.4266
5.0	0.2340	0.1916	0.1600	0.1284	0.2859	2.2669	0.2859
100	0.2019	0.1999	0.1981	0.1960	0.2040	2.3218	0.2040

Entropy rises monotonically with T , climbing toward its ceiling $\log_2 5 = 2.3219$ bits (the uniform distribution over 5 tokens). Temperature is a direct dial on entropy.

Mini-example · why halving T squares the odds

Between **mat** and **cat** the logit gap is $\Delta = 3 - 2 = 1$. The odds $p_{\text{mat}}/p_{\text{cat}}$ are exactly $e^{\Delta/T}$:

$$T=1 : e^1 = 2.72, \quad T=0.5 : e^2 = 7.39, \quad T=2 : e^{0.5} = 1.65.$$

Halving T *squares* the odds ratio ($2.72^2 = 7.39$); doubling T takes its square root. Check against the table: at $T=0.5$, $0.8643/0.1170 = 7.39$. ✓

3 The two limits

$T \rightarrow 0^+$ — **greedy** / $\arg \max$. Let $m = \arg \max_i z_i$ be the unique top logit. For any other j , the gap $z_j - z_m < 0$, so

$$\frac{p_j}{p_m} = e^{(z_j - z_m)/T} \xrightarrow{T \rightarrow 0^+} e^{-\infty} = 0.$$

All mass collapses onto the single best token: $\mathbf{p} \rightarrow (0, \dots, 1, \dots, 0)$. This is *greedy decoding* — deterministic, repetitive, safe.

$T \rightarrow \infty$ — **uniform**. Every $z_i/T \rightarrow 0$, so $e^{z_i/T} \rightarrow 1$ and

$$p_i \rightarrow \frac{1}{\sum_j 1} = \frac{1}{K}.$$

For $K = 5$ that is 0.2 each — exactly the $T = 100$ row above (0.2019, ...), already indistinguishable from uniform. Maximum diversity, maximum nonsense.

Intuition

Useful generation lives *between* these poles. $T \approx 0.0$ – 0.3 : factual Q&A, code, extraction — you want the model’s top pick. $T \approx 0.7$ – 1.0 : chat and writing — enough spread to be fluent and non-robotic. $T \gtrsim 1.3$: brainstorming, where you *want* the long tail. Temperature does not make the model smarter or dumber; it trades determinism for variety.

4 Relationship to sampling

Temperature reshapes the distribution; a *sampler* then draws from it. The two compose: temperature is usually applied *first*, and truncation (top- k , top- p , PDF #3) is applied to the tempered distribution. Lower T shrinks the tail that truncation would otherwise have to cut — which is why very low T makes top- p almost a no-op, and high T makes it essential. Greedy decoding is simply the $T \rightarrow 0$ limit, so it needs no sampler at all.

Equation cheat-sheet

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}, \quad \frac{p_a(T)}{p_b(T)} = e^{(z_a - z_b)/T}$$

$$T \rightarrow 0^+ : \mathbf{p} \rightarrow \text{one-hot}(\arg \max z), \quad T \rightarrow \infty : p_i \rightarrow \frac{1}{K}, \quad H(T) \text{ increases with } T \uparrow \log_2 K$$

Appendix A · Reference implementation

```
import numpy as np
```

```
def softmax_T(z, T=1.0):
    z = np.asarray(z, float) / T
    z = z - z.max()
    e = np.exp(z)
    return e / e.sum()
```

```
def entropy_bits(p):
    p = p[p > 0]
    return -(p * np.log2(p)).sum()
```

```
z = np.array([2.0, 1.0, 0.1, -1.0, 3.0])
for T in [0.5, 1.0, 2.0, 5.0, 100.0]:
    p = softmax_T(z, T)
    print(T, np.round(p, 4), round(entropy_bits(p), 4))
# 0.5  [.117  .0158 .0026 .0003 .8643]  0.6645
# 2.0  [.2587 .1569 .1001 .0577 .4266]  2.0181
```

Internal learning material. Numbers reproducible from the appendix code. v1.0